

宁波大学
NINGBO UNIVERSITY

网络集成实习

实验设计报告

姓名：	赵瑞杰
学号：	216001944
课程名称：	网络集成实习
所在学院：	信息科学与工程学院
专业班级：	21 级计算机科学与技术 1 班
任课教师：	陈海明

实验名称	实验三 网络系统服务——Web 服务创建

一、实验的目的与要求：

1.1 实验目的：

用 C++ Socket 编程实现一个 Web 服务器（端口号 8080 或其它）

1.2 实验要求：

1. 该 Web 服务器在一台主机上运行，支持多台主机同时访问（多线程）
2. 有服务开启和关闭的开关
3. 能够返回指定主页（index.html），页面可以是静态的内容，也可以包括实现 HTTP 的四个操作的动态交互。

1.3 实验测试方法：

在同组其他主机上，打开浏览器，输入 http://主机 IP 地址：8080 或指定的端口号，浏览器上显示主页，完成实验报告，要求给出实现的方法的具体说明和测试结果。

二、实验的设备及软件：

装配 Windows 操作系统的 PC 机多台，支持 C++编程的 IDE 等，Windows Sockets API (Winsock)。

三、系统设计的原理和流程图：

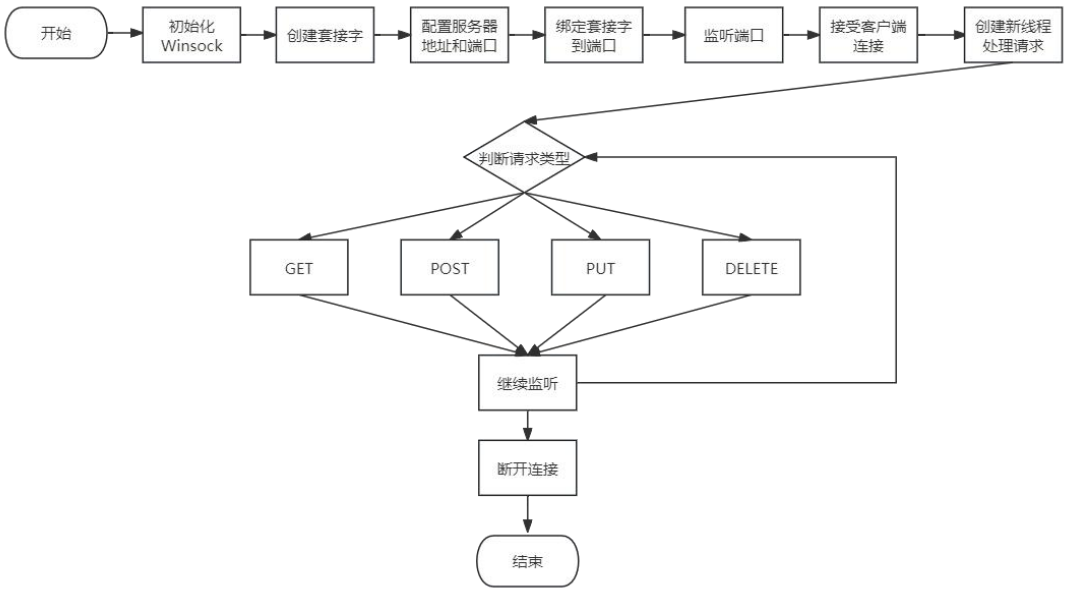
3.1 实验原理：

本次实验中，我们使用 C++结合适用于 Windows 操作系统的 Winsock2 编程接口来实现 Socket 编程。具体实验原理如下：

1. 套接字（Socket）：套接字是网络通信的基础。在 Windows 系统中，套接字通过套接字句柄来表示。套接字可以分为流套接字（使用 TCP 协议）和数据报套接字（使用 UDP 协议）。流套接字提供可靠的双向数据流，而数据报套接字则提供无连接的、尽最大努力交付的数据传输方式。
2. 创建套接字：通过调用 socket()函数来创建新的套接字。这个函数需要指定三个参数：地址族、套接字类型和协议。
3. 绑定：使用 bind()函数将创建的套接字绑定到特定的 IP 地址和端口上。这一步通常在服务器端执行，以便服务器可以在指定的地址和端口上监听客户端的连接请求。
4. 监听：通过调用 listen()函数将服务器套接字设置为监听模式，准备接受客户端连接。该函数需要指定监听队列的最大长度。
5. 接受连接：服务器端通过 accept()函数接受客户端的连接请求。该函数会阻塞程序运行，直到有客户端发起连接。
6. 客户端连接：客户端通过调用 connect()函数尝试连接到服务器的套接字。需要提供服务器的 IP 地址和端口号以完成连接。
7. 数据传输：一旦连接建立，客户端和服务器可以使用 send()和 recv()函数进行数据传输。对于 UDP 套接字，可以使用 sendto()和 recvfrom()函数。
8. 关闭连接：通信结束后，调用 close()函数关闭套接字。对于 TCP 连接，这会触发 TCP 协议的四次挥手过程，确保连接的双方都能正常关闭。
9. 多线程或非阻塞 I/O：为了提高服务器的性能和响应速度，通常使用多线程或非阻塞 I/O 来处理多客户端连接。多线程方式通过创建新线程来处理每个客户端的请求，非阻塞 I/O 则允许程序在等待 I/O 操作时继续执行其他任务，提高并发处理能力。

3.2 实验流程图：

本次实验过程中编写的服务器端 C++代码的具体流程如下图所示。



四、实验步骤和内容

4.1 服务器端代码

```
#include<stdio.h>
#include<iostream>
#include<WinSock2.h>
#pragma comment(lib,"ws2_32.lib")

#include<process.h>
using namespace std;
#pragma warning(disable : 4996)

unsigned WINAPI RequestHandler(void* arg);
int merror(int redata, int error, const char* showinfo);
void sendhtml(SOCKET s, char* filename);
void sendResponse(SOCKET s, const char* method);

int main() {
    DWORD dwThreadTD;

    printf("welcome to my web server!\n");

    // 初始化 Winsock 库
    WSADATA wsdata;
    int is_ok = WSAStartup(MAKEWORD(2, 2), &wsdata);
    merror(is_ok, WSAEINVAL, "申请 socket 失败！");
```

```
    // 创建服务器 socket
    SOCKET server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    merror(server, INVALID_SOCKET, "创建 socket 失败");

    // 设置服务器地址结构
    SOCKADDR_IN seraddr;
    seraddr.sin_family = AF_INET;
    seraddr.sin_port = htons(8080);
    seraddr.sin_addr.s_addr = inet_addr("192.168.43.170");

    // 绑定 socket
    is_ok = bind(server, (SOCKADDR*)&seraddr, sizeof(seraddr));
    merror(is_ok, SOCKET_ERROR, "socket 绑定失败！");

    // 监听连接
    is_ok = listen(server, 5);
    merror(is_ok, SOCKET_ERROR, "监听失败！");

    SOCKADDR_IN claddr;
    int cllen = sizeof(claddr);

    while (1) {
        // 接受客户端连接
        SOCKET client = accept(server, (SOCKADDR*)&claddr, &cllen);
        merror(client, INVALID_SOCKET, "连接失败！");

        // 创建线程处理客户端请求
        HANDLE hTread = (HANDLE)_beginthreadex(NULL, 0, RequestHandler, (void*)client,
0, (unsigned*)&dwThreadTD);
    }

    // 关闭服务器 socket
    closesocket(server);
    WSACleanup();

    getchar();
    return 0;
}

// 错误处理函数
int merror(int redata, int error, const char* showinfo) {
```

```
    if (redata == error) {
        perror(showinfo);
        getchar();
        return -1;
    }
    return 0;
}

// 请求处理函数
unsigned WINAPI RequestHandler(void* arg) {
    SOCKET client = (SOCKET)arg;
    char buffer[1024] = { 0 };

    // 接收客户端数据
    recv(client, buffer, 1024, 0);
    cout << "接收到的数据：" << buffer << endl;

    // 根据请求类型发送不同响应
    if (strstr(buffer, "GET") == buffer) {
        sendResponse(client, "GET");
    }
    else if (strstr(buffer, "POST") == buffer) {
        sendResponse(client, "POST");
    }
    else if (strstr(buffer, "PUT") == buffer) {
        sendResponse(client, "PUT");
    }
    else if (strstr(buffer, "DELETE") == buffer) {
        sendResponse(client, "DELETE");
    }

    // 关闭客户端 socket
    closesocket(client);
    return 0;
}

// 发送 HTML 页面
void sendhtml(SOCKET s, char* filename) {
    char protocol[] = "HTTP/1.0 200 OK\r\n";
    char serName[] = "Server: simple web server\r\n";
    char cntLen[] = "Content-length: 2048\r\n";
    char cntType[100];
```

```
    sprintf_s(cntType, "Content-type: %s\r\n\r\n", "text/html");

    send(s, protocol, strlen(protocol), 0);
    send(s, serName, strlen(serName), 0);
    send(s, cntLen, strlen(cntLen), 0);
    send(s, cntType, strlen(cntType), 0);

    // 打开 HTML 文件并发送其内容
    FILE* pfile = fopen(filename, "r");
    if (pfile == NULL) {
        cout << "can not open the file!" << endl;
        return;
    }
    char temp[1024] = "";
    do {
        fgets(temp, 1024, pfile);
        send(s, temp, strlen(temp), 0);
    } while (!feof(pfile));

    fclose(pfile);
}

// 发送响应信息
void sendResponse(SOCKET s, const char* method) {
    char protocol[] = "HTTP/1.0 200 OK\r\n";
    char serName[] = "Server: simple web server\r\n";
    char cntType[] = "Content-type: text/plain\r\n\r\n";
    char message[100];

    // 构建响应消息
    sprintf_s(message, "You sent a %s request.\n", method);

    send(s, protocol, strlen(protocol), 0);
    send(s, serName, strlen(serName), 0);
    send(s, cntType, strlen(cntType), 0);
    send(s, message, strlen(message), 0);
}
```

代码说明：

1. 初始化和服务器启动

- 使用 WSAStartup 函数初始化 Winsock 库，确保正确使用 Windows 套接字 API。
- 创建服务器 socket，使用 TCP 协议。
- 绑定服务器 socket 到特定的 IP 地址和端口（这里使用了 IP 地址 192.168.43.170 和端

口 8080）。

- 监听连接，等待客户端连接请求。

2. 请求处理循环
   - 进入一个无限循环，持续接受客户端连接。
   - 每当有新的客户端连接请求到来时，接受连接并为该连接创建一个新的线程来处理请求，以保持服务器的并发性能。
3. 错误处理
   - 定义了一个错误处理函数 merror，用于检查函数返回值并在出现错误时输出错误信息并终止程序执行。
4. 请求处理函数
   - RequestHandler 函数用于处理客户端请求，它首先接收客户端发送的数据，然后根据请求类型发送相应的响应消息给客户端。
   - 当收到 GET 请求时，调用 sendResponse 函数发送一条 GET 请求的响应消息；当收到 POST、PUT、DELETE 请求时，分别发送对应类型的响应消息。
5. 发送 HTML 页面和响应信息
   - sendhtml 函数用于向客户端发送 HTML 页面，它打开指定的 HTML 文件并逐行发送文件内容给客户端。
   - sendResponse 函数用于发送响应信息，根据传入的请求类型构建相应的响应消息并发送给客户端。

4.2 web 服务代码

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Success Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f0f8ff;
            color: #333;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }

        .container {
            text-align: center;
            background: #fff;
            padding: 2rem;
```

```
        border-radius: 10px;
        box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    }

    h2 {
        color: #4CAF50;
        font-size: 2rem;
        margin-bottom: 1rem;
    }

    p {
        font-size: 1.2rem;
        margin: 0;
    }

    button {
        margin: 0.5rem;
        padding: 0.7rem 1.5rem;
        font-size: 1rem;
        cursor: pointer;
        border: none;
        border-radius: 5px;
        transition: background-color 0.3s ease;
    }

        button:nth-child(3) {
            background-color: #ff9800;
            color: white;
        }

        button:nth-child(5) {
            background-color: #00ccff;
            color: white;
        }

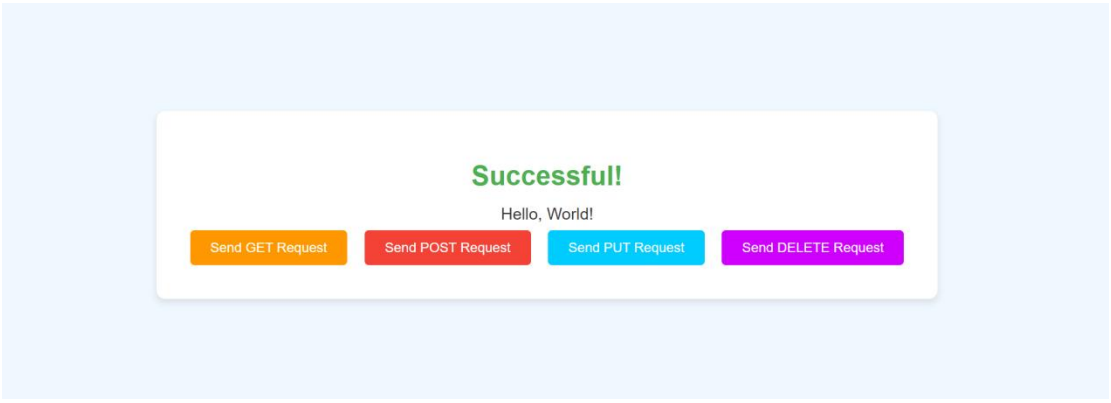
        button:nth-child(6) {
            background-color: #d000ff;
            color: white;
        }

        button:nth-child(4) {
            background-color: #f44336;
```

```
                color: white;
            }
    </style>
    <script>
        async function sendRequest(method) {
            const response = await fetch('http://192.168.43.170:8080', {
                method: method
            });
            const text = await response.text();
            console.log(`Response for ${method}:`, text);
        }
    </script>
</head>
<body>
    <div class="container">
        <h2>Successful!</h2>
        <p>Hello, World!</p>
        <button class="get-button" onclick="sendRequest('GET')">Send GET Request</button>
        <button class="post-button" onclick="sendRequest('POST')">Send POST Request</button>
        <button class="put-button" onclick="sendRequest('PUT')">Send PUT Request</button>
        <button class="delete-button" onclick="sendRequest('DELETE')">Send DELETE Request</button>
    </div>
</body>
</html>
```

五、测试结果

5.1 主页面展示



5.2 启动 web 服务，并持续监听端口

① 输入 http://192.168.43.170:8080/，返回 index.html，控制台打印信息如下：

```
接收到的数据：GET / HTTP/1.1
Host: 192.168.43.170:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
```

② 点击 Send GET Request，发送 GET 请求：

```
接收到的数据：GET / HTTP/1.1
Host: 192.168.43.170:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Referer: http://192.168.43.170:8080/
```

③ 点击 Send POST Request，发送 POST 请求：

```
接收到的数据：POST / HTTP/1.1
Host: 192.168.43.170:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Content-Length: 0
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Origin: http://192.168.43.170:8080
Referer: http://192.168.43.170:8080/
```

④ 点击 Send PUT Request，发送 PUT 请求：

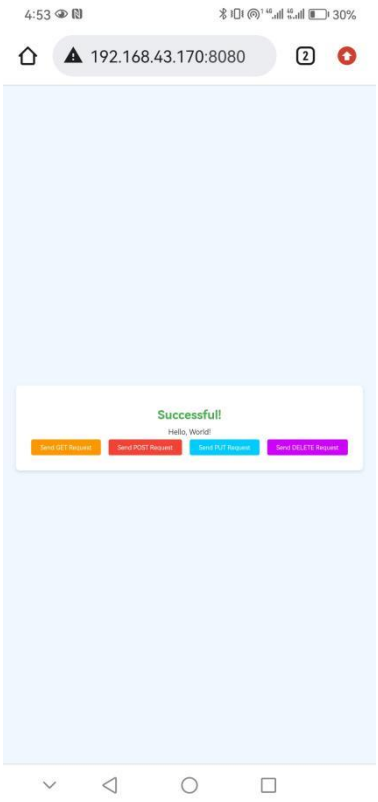
```
接收到的数据：PUT / HTTP/1.1
Host: 192.168.43.170:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Content-Length: 0
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Origin: http://192.168.43.170:8080
Referer: http://192.168.43.170:8080/
```

⑤ 点击 Send DELETE Request，发送 DELETE 请求：

```
接收到的数据：DELETE / HTTP/1.1
Host: 192.168.43.170:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Origin: http://192.168.43.170:8080
Referer: http://192.168.43.170:8080/
```

5.3 其他主机上访问服务器

在相同 WLAN 下使用个人手机访问服务器（ip 地址为 192.168.43.170:8080）：



点击结果和第二部分相同。

六、实验结论

本次实验的主要任务是使用 C++的 Socket 编程实现一个 Web 服务器，并能够支持多台主机的同时访问。通过本次实验，我深入理解了 Socket 编程的基本原理和实现方法。首先，在服务器端，我们使用 Winsock2 库进行 Socket 编程，创建一个监听特定端口的服务器，能够处理多种 HTTP 请求（GET、POST、PUT、DELETE）。通过线程池实现多线程处理，提高了服务器的并发性能。在客户端，我们设计了一个简单的 HTML 页面，并通过 JavaScript 发送各种 HTTP 请求，从而验证服务器的处理能力。

实验过程中，遇到的主要挑战在于多线程的实现和 Socket 的稳定性。通过查阅相关资料和不断调试，最终成功实现了服务器的多线程处理，并确保了 Socket 的稳定连接。在实验中，我体会到了网络编程的复杂性和趣味性，理解了 TCP/IP 协议的工作原理以及 HTTP 请求处理的基本流程。通过在不同设备上测试服务器的响应情况，验证了程序的正确性和稳定性。本次实验不仅提升了我的 C++编程能力，还让我对网络通信有了更深的理解，对未来深入学习和应用网络编程打下了坚实的基础。